

Módulo 1: Introdução à Linguagem Python e Configuração do Ambiente

- **História e Conceito:** A linguagem Python foi desenvolvida por Guido van Rossum no final dos anos 1980 na Holanda. Trata-se de uma linguagem de alto nível, interpretada, de código aberto, portátil e multiparadigma, suportando programação procedural, orientada a objetos e funcional. É dinamicamente tipada e utiliza a indentação visual por padrão para delimitar blocos de código.
- **Aplicações na Análise de Dados:** Python é amplamente utilizado em inteligência artificial, *machine learning*, ciência de dados, visualização de dados, automação e desenvolvimento web.
- **Configuração do Ambiente:** O interpretador oficial é baixado em python.org, sendo essencial marcar a opção de adicionar o executável à variável de ambiente PATH durante a instalação. Para o desenvolvimento, utilizam-se editores e IDEs como VS Code, PyCharm, IDLE ou Jupyter Lab.
- **Modo Interativo vs. Scripts:** O modo interativo (REPL) executa comandos instantaneamente e exibe o prompt >>>. Já os scripts são salvos em arquivos com extensão .py e executados sequencialmente pelo interpretador.

Módulo 2: Fundamentos da Linguagem – Variáveis, Tipos de Dados e Operadores

- **Variáveis e Memória:** Uma variável é um local reservado na memória do computador rotulado por um nome que armazena um valor.
- **Tipos de Dados Primitivos:**
 - int: Números inteiros.
 - float: Números reais com casas decimais (ponto flutuante).
 - str: Cadeias de caracteres (texto), delimitadas por aspas.
 - bool: Valores lógicos booleanos (True ou False).
 - complex: Números complexos contendo parte real e imaginária.
- **Verificação e Conversão de Tipos (Casting):** O tipo de um dado pode ser consultado com type() ou checado com isinstance(). A conversão explícita entre tipos é realizada pelas funções int(), float() e str().
- **Operadores Aritméticos:** Adição (+), Subtração (-), Multiplicação (*), Divisão real (/), Divisão inteira (//), Módulo/Resto (%) e Potenciação (**).
- **Operadores Relacionais e Lógicos:** Comparações retornam booleanos através de ==, !=, >, <, >=, <=, is e is not. Os operadores lógicos and, or e not avaliam expressões lógicas com comportamento de avaliação por curto-circuito.

Módulo 3: Estruturas de Controle de Fluxo – Condicionais e Repetições

- **Estruturas Condicionais:** Permitem desviar o fluxo de execução com base em testes lógicos.
 - if: Executa o bloco se a condição for verdadeira.
 - else: Executa o bloco caso a condição do if seja falsa.
 - elif: Permite testar múltiplas condições encadeadas em sequência.
- **Indentação de Blocos:** Blocos de código em Python são definidos rigorosamente por recuos de espaço (geralmente 4 espaços) após o caractere de dois pontos (:).
- **Estruturas de Repetição:**
 - while: Executa repetidamente um bloco enquanto uma condição lógica permanecer verdadeira. Exige controle explícito da condição para evitar laços infinitos.
 - for: Itera sobre sequências de elementos (como listas, tuplas ou strings). É frequentemente combinado com a função range(início, fim, passo) para gerar sequências numéricas.

- **Controle de Laços:** A instrução `break` interrompe e encerra o laço imediatamente; a instrução `continue` interrompe a iteração atual e salta para a próxima.
- **Laços Encadeados:** Podem-se aninhar laços dentro de outros laços para processar matrizes e estruturas bidimensionais.

Módulo 4: Estruturas de Dados Sequenciais e Coleções

- **Strings (str):** Sequências imutáveis de caracteres. Suportam indexação por posição (iniciando em 0), fatiamento (*slicing* [início:fim:passo]), concatenação (+) e diversos métodos de manipulação.
- **Listas (list):** Coleções ordenadas e mutáveis delimitadas por colchetes []. Suportam adição de elementos (`append()`, `insert()`), remoção (`pop()`, `remove()`), ordenação (`sort()`, `sorted()`) e funções utilitárias (`len()`, `sum()`, `min()`, `max()`).
- **Tuplas (tuple):** Sequências ordenadas e imutáveis delimitadas por parênteses (). São utilizadas para proteger dados que não devem ser alterados e para desempacotamento de variáveis.
- **Dicionários (dict):** Estruturas mutáveis que armazenam dados no formato de pares chave-valor delimitados por chaves {}. As chaves devem ser únicas e imutáveis. Suportam consulta eficiente através das chaves e métodos como `keys()`, `values()`, `items()` e `get()`.
- **Conjuntos (set):** Coleções não ordenadas de elementos únicos delimitadas por chaves {} ou criadas com `set()`. Não admitem duplicatas e realizam operações matemáticas de união (`|`), interseção (`&`), diferença (`-`) e diferença simétrica (`^`).

Módulo 5: Funções, Escopo, Módulos e Recursos Avançados

- **Definição de Funções:** Criadas com a palavra-chave `def`, recebem parâmetros e podem retornar valores através do comando `return`. Funções sem `return` explícito retornam `None`.
- **Tipos de Parâmetros:** Podem ser posicionais, nomeados ou possuir valores padrão (parâmetros opcionais).
- **Escopo de Variáveis:** Variáveis criadas dentro de funções são locais. Para modificar variáveis globais dentro de uma função, declara-se explicitamente a instrução `global`.
- **Recursividade:** Técnica em que uma função chama a si mesma, exigindo a definição rigorosa de um caso base e de um caso recursivo.
- **Módulos e Pacotes:** Permitem a organização e reuso de código distribuído em arquivos `.py`. São importados com `import` ou `from ... import`. A instrução `if __name__ == '__main__':` garante a separação entre execução direta e importação. O gerenciador de pacotes PIP permite instalar bibliotecas externas do PyPI.
- **Recursos Avançados de Programação:**
 - **Expressões Lambda:** Funções anônimas simplificadas de uma única linha.
 - **Funções Utilitárias:** `map()` aplica uma função a cada elemento de um iterável; `filter()` filtra elementos com base em uma condição lógica.
 - **List Comprehensions:** Sintaxe concisa para criar novas listas aplicadas sobre iteráveis.
 - **Generators:** Utilizam a instrução `yield` para produzir valores sob demanda (lazy evaluation), economizando memória do sistema.

Módulo 6: Tratamento de Exceções, Manipulação de Arquivos e POO

- **Tratamento de Exceções:** Previne a interrupção abrupta do programa diante de erros em tempo de execução. Utiliza o bloco `try` para o código vulnerável, except para capturar

a exceção, else para execução na ausência de erros e finally para código executado obrigatoriamente no final. Erros podem ser lançados forçadamente com raise.

- **Manipulação de Arquivos de Texto:** Arquivos no disco oferecem persistência secundária de dados. Abrem-se arquivos com a função `open(nome, modo, encoding="utf-8")` nos modos leitura ('r'), escrita ('w') ou anexo ('a'). A declaração `with` (gerenciador de contexto) garante o fechamento automático do arquivo após o uso.
- **Programação Orientada a Objetos (POO):** Modelo de organização de software que representa entidades do mundo real em classes e objetos.
 - **Classe e Objeto:** A classe é o modelo/molde; o objeto é a instância da classe criada na memória.
 - **Construtor e Atributos:** O método `__init__()` inicializa o objeto; a referência `self` acessa os atributos e métodos internos.
 - **Encapsulamento:** Atributos privados são prefixados com duplo sublinhado (`__`) e acessados por métodos Getters e Setters.
 - **Herança e Polimorfismo:** Uma classe filha herda atributos e métodos de uma classe pai usando a instrução `super()`. O polimorfismo permite redefinir métodos nas classes filhas para adaptar comportamentos.

Módulo 7: Manipulação do Sistema Operacional, Banco de Dados e Ecossistema de Dados

- **Interação com o S.O. (os e os.path):** Permite navegar e manipular arquivos/diretórios com comandos como `getcwd()`, `chdir()`, `mkdir()`, `makedirs()`, `rename()`, `remove()`, `rmdir()`, `path.join()`, `path.exists()` e `path.splitext()`.
- **Bancos de Dados Relacionais (sqlite3):** O módulo `sqlite3` permite integrar bancos de dados locais permanentes. As operações envolvem conexão (`sqlite3.connect()`), criação de cursores (`cursor()`) e execução de comandos SQL como `CREATE TABLE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE` e `JOIN`.
- **Visão Geral do Ecossistema de Análise de Dados:** Integração com bibliotecas para tratamento numérico (NumPy), manipulação tabular e séries temporais (Pandas), gráficos (Matplotlib), raspagem web (BeautifulSoup, urllib) e estruturas de troca de dados (JSON e XML).